INVENTRA

Technical & Architecture Documentation

Web app (React + Vite + TypeScript + Supabase) and companion mobile app (Expo / React Native)
Generated 2026-08-11 — compiled directly from the project's source code

---

Every statement in this document was verified by reading the actual files in the repository.

Table of Contents

1. Overview & Architecture

INVENTRA is a stock/inventory and parcel-tracking system for a T-shirt/apparel business. It consists of two client applications that both talk directly to a shared Supabase backend — there is no custom REST/API server in the current codebase.

Web frontend	`frontend/` — React 19 + Vite 8 + TypeScript, deployed as a static SPA
Mobile app	`mobile/` — Expo (React Native) 57, a lightweight companion app (dashboard stats, product list, barcode scanner UI)
Backend	Supabase: PostgreSQL (with Row Level Security), Supabase Auth, Supabase Storage, one Supabase Edge Function
Data access pattern	Both clients hold the Supabase anon key and query PostgREST directly via `@supabase/supabase-js`. Authorization is enforced entirely by Postgres RLS policies, not by an application server.

The root `README.md` explicitly labels this "INVENTRA (Supabase Edition)" and states it is "frontend-only." The root `.gitignore` still references a `backend/` Python/Django path (`backend/__pycache__/`, `backend/db.sqlite3`), but no such folder exists in the repository today — this is a leftover from an earlier architecture. See Section 13 for the stale documentation this history left behind.

2. Repository / Folder Structure

Full structure as read from the working tree (excluding `node_modules`, `.git`, `dist`):

- `README.md` — Top-level project overview
- `.env` — Present, empty (0 bytes), unused
- `database/SCHEMA.md` — STALE, describes a Django-era schema (see Sec. 13)
- `docs/API_DOCUMENTATION.md` — Confirms no REST/Django server; points to Supabase
- `docs/DEPLOYMENT_GUIDE.md` — Deployment steps (Supabase + Vercel + Expo)
- `docs/PRODUCTION_CHECKLIST.md` — Pre-launch checklist
- `docs/SWAGGER.md` — Confirms no Swagger/OpenAPI surface exists anymore
- `docs/postman_collection.json` — Empty item list; notes Supabase SDK is used directly
- `scripts/dev-start.ps1` — Local dev helper script (PowerShell)
- `supabase/APPLY_ALL.sql` — Consolidated, idempotent schema, source of truth
- `supabase/README.md` — Supabase setup steps
- `supabase/migrations/` — 11 incremental SQL migration files (see Sec. 8)
- `supabase/functions/low-stock-alert/index.ts` — One Supabase Edge Function (see Sec. 9)

frontend/ — React + Vite + TypeScript web app

- `package.json` / `package-lock.json`
- `vite.config.ts`, `tailwind.config.ts`, `tsconfig*.json`
- `vercel.json` — SPA rewrite rule for Vercel
- `components.json` — shadcn/ui config (slate base, CSS variables)

- `.env` / `.env.example` — `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`
- `public/` — logo.png, favicon.svg, icons.svg, skeleton-hero.png, inventra-hero.mp4
- `src/main.tsx` — App entry: QueryClientProvider, ThemeProvider, AuthProvider, BrowserRouter
- `src/App.tsx` — Route table (see Sec. 5)
- `src/App.css`, `src/index.css`
- `src/app/ProtectedRoute.tsx` — Auth gate for the main route tree
- `src/app/ChunkErrorBoundary.tsx` — Recovers from stale-deploy chunk-load errors
- `src/layouts/AppLayout.tsx` — Shell: desktop sidebar, mobile header + bottom nav, notification bell
- `src/contexts/AuthContext.tsx` — Session/user state, login/logout/profile (see Sec. 6)
- `src/contexts/ThemeContext.tsx` — Light/dark theme, persisted to localStorage
- `src/pages/` — 25 route-level page components (see Sec. 10.1)
- `src/services/` — 6 Supabase data-access modules (see Sec. 10.2)
- `src/hooks/` — 4 custom hooks (see Sec. 10.3)
- `src/lib/` — 7 utility modules incl. `supabase.ts` client (see Sec. 10.4)
- `src/components/dashboard/` — StatCard, TrendChart
- `src/components/shared/` — EmptyState, GoogleIcon, InventraSearchGhostHero, Skeleton, WarehouseScene
- `src/components/ui/` — button, card, confirm-dialog, input (shadcn-style primitives)
- `src/types/index.ts` — Shared domain TypeScript types
- `src/assets/` — hero.png, react.svg, vite.svg (Vite template leftovers)

mobile/ — Expo (React Native) companion app

- `package.json`, `app.json`, `babel.config.js`, `tsconfig.json`
- `.env.example` — `EXPO_PUBLIC_SUPABASE_URL`, `EXPO_PUBLIC_SUPABASE_ANON_KEY`
- `AGENTS.md` / `CLAUDE.md` — Note: pins Expo docs to v57.0.0
- `App.tsx` — Bottom-tab navigator (Dashboard / Products / Scanner)
- `index.ts` — Expo entry point
- `assets/` — App icons, splash
- `src/screens/` — DashboardScreen, ProductsScreen, ScannerScreen
- `src/services/api.ts` — Supabase client for the mobile app

3. Tech Stack & Dependencies

3.1 Frontend — frontend/package.json

Package	Version	Role
react / react-dom	^19.2.8	UI framework
vite	^8.2.0	Build tool / dev server
typescript	~6.0.2	Type checking (build runs `tsc -b` before `vite build`)
react-router-dom	^7.18.2	Client-side routing (`BrowserRouter`)
@supabase/supabase-js	^2.57.4	Supabase client SDK (Auth, Postgres, Storage)
@tanstack/react-query	^5.101.4	Server-state caching/invalidation for Supabase reads
react-hook-form	^7.84.0	Form state
zod	^4.4.3	Schema validation, paired with @hookform/resolvers
framer-motion	^12.43.0	Animation (login hero, stat cards, transitions)
tailwindcss	^4.3.3	Utility-first CSS (+ tailwindcss-animate, @tailwindcss/postcss)
class-variance-authority / clsx / tailwind-merge	—	Class-name composition for UI primitives (`lib/utils.ts`)
lucide-react	^1.28.0	Icon set
recharts	^3.10.1	Charts (dashboard trend chart)
sonner	^2.0.7	Toast notifications
date-fns	^4.4.0	Date formatting
jspdf	^4.2.1	Client-side PDF generation (order receipts, address exports)
tesseract.js	^7.0.0	Client-side OCR (extracts shipping address text from an uploaded screenshot on the Enquiry page)
oxlint (dev)	^1.75.0	Linting (`npm run lint`)

Scripts (`frontend/package.json`): `dev` → `vite`, `build` → `tsc -b && vite build`, `lint` → `oxlint`, `preview` → `vite preview`.

3.2 Mobile — mobile/package.json

Package	Version	Role
expo	~57.0.9	App runtime/toolchain
react-native	0.86.2	UI framework
react	19.2.3	UI framework
@supabase/supabase-js	^2.57.4	Direct Supabase access (same backend as web)
@react-navigation/native, @react-navigation/bottom-tabs	—	Bottom-tab navigation (Dashboard / Products / Scanner)
expo-camera	^7.x	Camera access for the barcode/QR scanner screen
react-native-reanimated, react-native-gesture-handler, react-native-screens, react-native-safe-area-context	~57.0.3	Navigation/animation infrastructure required by React Navigation

Scripts: `start` → `expo start`, plus `android`, `ios`, `web` variants.

`mobile/AGENTS.md` (linked from `CLAUDE.md`) contains a note that Expo "HAS CHANGED" and instructs referring to the versioned docs at docs.expo.dev/versions/v57.0.0/ before writing mobile code — worth keeping in mind if extending the mobile app.

3.3 Root of repository

There is no root `package.json`. A root `package-lock.json` exists but is an empty lockfile (`"packages": {}`), not functionally used. The frontend and mobile apps are independent npm projects.

4. Environment Variables

4.1 Frontend — frontend/.env / .env.example

Variable	Purpose
VITE_SUPABASE_URL	Supabase project URL, read in `frontend/src/lib/supabase.ts`
VITE_SUPABASE_ANON_KEY	Supabase anon/public key, same file

If either is missing, `hasSupabaseConfig` (in `lib/supabase.ts`) becomes `false`: the app falls back to a hardcoded demo user (`AuthContext.tsx`, `demoUser`) with no real authentication, and login/register controls are disabled in the UI with a visible warning banner. The Supabase client itself is still constructed with placeholder values (`https://placeholder.supabase.co`) so the app doesn't crash at import time.

4.2 Mobile — mobile/.env.example

Variable	Purpose
EXPO_PUBLIC_SUPABASE_URL	Supabase project URL, read in `mobile/src/services/api.ts`
EXPO_PUBLIC_SUPABASE_ANON_KEY	Supabase anon/public key, same file

Unlike the web client, the mobile Supabase client throws at import time if these are missing (no placeholder fallback), and is configured with `persistSession: false` and `autoRefreshToken: false` — the mobile app currently has no persisted auth session and re-authenticates (or runs unauthenticated against RLS-protected tables) on every app start.

4.3 Supabase Edge Function secrets

Variable	Purpose
SUPABASE_URL	Read by `supabase/functions/low-stock-alert/index.ts` via `Deno.env.get`
SUPABASE_SERVICE_ROLE_KEY	Same function — used to bypass RLS and scan all products' stock levels

Per `supabase/README.md`, these are set as Supabase function secrets, never shipped to any client.

4.4 Root .env

A file exists at the repository root (`/.env`) but is 0 bytes — present, empty, unused by any code path found.

5. Routing

Defined entirely in `frontend/src/App.tsx` using react-router-dom v7's `<Routes>`, with every page lazy-loaded via `React.lazy` inside a `<Suspense>` boundary. The router is `BrowserRouter`, mounted in `main.tsx`; `frontend/vercel.json` rewrites all paths to `/index.html` so deep links work on Vercel's static hosting.

5.1 Route table

Path	Component	Access
/login	LoginPage	PUBLIC
/ (index)	redirect to /home	PROTECTED
/home	HomePage	PROTECTED
/inventory	InventoryPage	PROTECTED
/enquiry	EnquiryPage	PROTECTED
/stock-up	StockUpPage	PROTECTED
/track	TrackPage	PROTECTED
/delivered-orders	DeliveredOrdersPage	PROTECTED
/order-status	OrderStatusPage	PROTECTED
/more	MorePage	PROTECTED
/management	ManagementPage	PROTECTED
/notifications	NotificationsPage	PROTECTED
/dashboard	DashboardPage	PROTECTED
/stock-in	redirect to /stock-up	PROTECTED
/parcels	redirect to /track	PROTECTED
/products	ProductsPage	PROTECTED
/distributors	DistributorManagementPage	PROTECTED
/settings	SettingsPage	PROTECTED
/profile	ProfilePage	PROTECTED
/about	AboutPage	PROTECTED
/privacy-policy	PrivacyPolicyPage	PROTECTED
/terms	TermsPage	PROTECTED
/support	SupportPage	PROTECTED
* (catch-all)	redirect to /home	PROTECTED

All protected routes are children of one `<ProtectedRoute><AppLayout /></ProtectedRoute>` element mounted at `/`, so they share the sidebar/bottom-nav shell from `layouts/AppLayout.tsx` and render into its `<Outlet />`.

5.2 Navigation menus (from AppLayout.tsx)

Mobile bottom nav	Home (/home), Dashboard (/dashboard), Track (/track), More Info (/more)
Desktop sidebar	Reports (/dashboard), Stock Up (/stock-up), Enquiry (/enquiry), Order Summary (/home), Track (/track), Delivered Orders (/delivered-orders), More Info (/more), Management (/management)

> **Orphaned pages:** seven page components exist in `frontend/src/pages/` but are not imported by `App.tsx` and have no route or nav link anywhere in the frontend — they are unreachable from the running app: `AllocateStockPage.tsx`, `AllocationHistoryPage.tsx`, `ScannerPage.tsx`, `StockInPage.tsx`, `SupplierManagementPage.tsx`, `ParcelTrackingPage.tsx`, `ModulePage.tsx`. Confirmed by grepping their component names across `frontend/src` — each only appears in its own defining file. Some functionality overlaps with routed pages (e.g. stock-out/allocation logic already lives in StockUpPage / EnquiryPage via `services/inventory.ts`), so these may be earlier iterations left in place rather than in-progress features — the code alone doesn't say which.

6. Authentication & Authorization

6.1 Client setup

`frontend/src/lib/supabase.ts` creates one supabase-js client with `persistSession: true`, `autoRefreshToken: true`, `detectSessionInUrl: true` (the last one is what makes OAuth-redirect and password-reset links work automatically).

6.2 Session state — contexts/AuthContext.tsx

A single `supabase.auth.onAuthStateChange` subscription is the sole source of truth for session state (it fires immediately on subscribe with the current session, then again on every sign-in/out/refresh). On any non-null session it calls `fetchProfile()` (`services/auth.ts`) to load the `profiles` row; if that fails, the code deliberately does not leave the app in a half-authenticated state — it resets `user` to null and `isAuthenticated` to false so `ProtectedRoute` correctly redirects to `/login`.

If `hasSupabaseConfig` is false (missing env vars), `AuthProvider` skips Supabase entirely and signs the app in as a static `demoUser`.

6.3 Supported sign-in methods

Method	Implementation	Status
Email + password	`supabase.auth.signInWithPassword` (`services/auth.ts` `login()`), wrapped in `retryOnNetworkError` (3 attempts, linear backoff, only retries transient `AuthRetryableFetchError`s)	Working — `mailer_autoconfirm: true` confirmed live on the project's Auth settings, so signup does not require email confirmation
Email + password sign-up	`supabase.auth.signUp()` (`services/auth.ts` `register()`), falls back to an immediate sign-in if signup returns no session	Working
Google OAuth	`supabase.auth.signInWithOAuth({ provider: "google" })` in `LoginPage.tsx`, redirecting to `{origin}/home`	Disabled at the Supabase project level. A direct query to `{SUPABASE_URL}/auth/v1/settings` returns `"google": false` under `external` — the frontend code is complete and correct, but the provider has not been turned on in Supabase Dashboard → Authentication → Providers, so clicking the button currently fails server-side before reaching Google.

6.4 Profile bootstrap

A Postgres trigger, `on_auth_user_created` (function `handle_new_user()`, in `supabase/migrations/20260801_init.sql`), fires after insert on `auth.users` and inserts a matching row into `public.profiles` — username falls back to the local part of the email, first/last name from `raw_user_meta_data` if present (populated for email/password signups; would be empty for Google sign-ins once that provider is enabled, since Google's metadata keys differ).

6.5 Route protection

`app/ProtectedRoute.tsx`: renders nothing while `isAuthReady` is false (avoids a flash of the login page during the initial session check), redirects to `/login` if not authenticated, otherwise renders its children.

6.6 Inactivity logout

`hooks/useInactivityLogout.ts`, wired into `AppLayout`: listens for mouse/keyboard/wheel/touch/scroll activity and signs the user out automatically after 30 minutes of inactivity (reschedule is throttled to once per 5s to avoid excessive timer churn).

6.7 Authorization model (database side)

There is no server-side authorization layer beyond Postgres. Every domain table has Row Level Security enabled with an "owner" policy of the form `using (auth.uid() = created_by)` (or `= user_id` for user-scoped tables) — see Section 7.3. This means each signed-in user only ever sees/edits their own data; there is no shared/team workspace concept, and the `profiles.role` column (default `'owner'`) is not read or branched on anywhere in the frontend code that was inspected.

7. Database Schema (Supabase / PostgreSQL)

Source of truth: `supabase/APPLY_ALL.sql` — a consolidated, idempotent concatenation of every migration in `supabase/migrations/`, meant to be pasted into the Supabase SQL Editor directly. All tables live in the `public` schema.

7.1 Tables

Table	Key columns	Notes
profiles	id (PK, = auth.users.id), username (unique), first_name, last_name, email (unique), mobile, avatar_url, role (default 'owner')	1:1 with an auth user; bootstrapped by trigger (Sec. 6.4)
app_settings	id, user_id, business_name (default 'INVENTRA'), company_logo, gst_number, dark_mode	Soft-delete via deleted_at
categories	id, name, description, status, created_by, updated_by	Unique (created_by, name)
brands	id, name, description, status, logo_url (added later), created_by, updated_by	Unique (created_by, name)
distributors	company_name, distributor_name, gst_number, mobile, whatsapp, email, address/city/state/pincode, notes, status	Owner-scoped CRUD entity
suppliers	supplier_name, company_name, gst, mobile, email, address/city/state, notes, status	Owner-scoped CRUD entity
products	name, category_id (FK), brand_id (FK), sku, barcode, qr_code (all nullable after a later migration), purchase_price, selling_price, gst, unit, description, status, image_url, minimum_stock, reorder_level	Unique (created_by, sku), (created_by, barcode), (created_by, qr_code). Soft-delete via deleted_at
product_sizes	product_id (FK, cascade delete), size, current_stock, available_stock, minimum_stock, maximum_stock, purchase_price, last_updated	Unique (product_id, size). current_stock / available_stock maintained only by the stock-entry trigger, never written directly by the app
stock_entries	product_id, product_size_id, distributor_id (nullable), supplier_id (nullable), invoice_number, quantity (check > 0), purchase_cost, transaction_type (check in stock_in / stock_out / adjustment / return), color, allocated_to, allocated_to_type, notes, received_date	The single ledger for every stock movement — stock in, stock out, and allocations are all rows here, distinguished by transaction_type
stock_history	stock_entry_id (FK, cascade), product_id, product_size_id, previous_stock, changed_stock, current_stock, event_type, notes	Append-only audit trail, written automatically by the stock trigger
parcels	parcel_id (unique), tracking_number (unique), barcode (unique), qr_code (unique), distributor_id/supplier_id (nullable), invoice_number, product_id, quantity, dispatch/arrival/delivery_date,	Used by ParcelTrackingPage / ScannerPage — both currently orphaned (Sec. 5.1) — and by services/analytics.ts

	current_location, current_status (default 'ordered'), assigned_person, remarks	
notifications	title, message, notification_type, is_read, user_id	Populated automatically by the low-stock trigger and by the edge function; read by NotificationsPage
activity_logs	user_id, module, action, metadata (jsonb)	Written by the stock-entry trigger on every insert; no UI currently reads this table
enquiries	order_id (auto-generated INV###, unique per created_by), customer_name, assigner_name, custom_requirement, combinations (jsonb), shipping_image_url, tracking_number, order_status (default 'New'), status_history (jsonb), delivery_date, hidden (default false), notes, extracted_address	Added in a later migration (20260804); this is the "order" entity used throughout HomePage/EnquiryPage/TrackPage/OrderStatusPage

7.2 Functions & triggers

Function / Trigger	Fires on	Behavior
set_current_timestamp_updated_at()	before update — attached to profiles, app_settings, categories, brands, distributors, suppliers, products, product_sizes, parcels, notifications, enquiries	Sets `updated_at = now()`
handle_stock_entry_after_insert()	after insert on stock_entries (trigger `trg_stock_entry_after_insert`)	`security definer`. Adds/subtracts quantity from `product_sizes.current_stock` (and mirrors into `available_stock`) depending on `transaction_type`; inserts a `stock_history` row; if the resulting stock is at or below that size's `minimum_stock`, inserts a Low Stock Alert row into `notifications` (type `out_of_stock` if ≤ 0, else `low_stock`); always inserts an `activity_logs` row.
handle_new_user()	after insert on auth.users (trigger `on_auth_user_created`)	`security definer`. Bootstraps the matching `profiles` row (Sec. 6.4)
set_enquiry_order_id()	before insert on enquiries (trigger `trg_enquiries_order_id`)	If `order_id` is blank, computes the next sequential INV### number scoped to that user by parsing the trailing digits of their existing order IDs

7.3 Row Level Security

RLS is enabled on every table listed above. Every policy is an owner-scoped `for all using (auth.uid() = created_by) with check (...)` (or `= user_id` / `= id` for notifications / app_settings / profiles) — one policy per table, no separate read/write/admin roles. There is no cross-user sharing: each authenticated user's data is fully isolated by `auth.uid()`.

7.4 Storage buckets

Bucket	Public	Used by	Write policy
product-images	Yes	services/inventory.ts (product photo upload)	Insert/update/delete require `auth.uid() is not null`; read is unrestricted
profile-photos	Yes	services/auth.ts uploadProfilePhoto()	Same pattern
enquiry-images	Yes	services/enquiries.ts createEnquiry() (shipping screenshot)	Same pattern
brand-logos	Yes	services/inventory.ts (brand logo upload, Management page)	Same pattern

All four Storage buckets are public and their read policies allow anyone with a file's URL to fetch it (no auth check on select) — this is intentional for serving images in the UI, but means uploaded product/profile/enquiry/brand images are not access-controlled beyond obscurity of the URL.

8. Migration History

11 files in `supabase/migrations/`, applied in filename (date) order. `supabase/APPLY_ALL.sql` is the concatenation of all of them and is the file actually meant to be run against a fresh project.

File	Change
20260801_init.sql	Initial schema: all core tables (profiles → activity_logs), triggers, RLS policies, product-images + profile-photos buckets
20260802_product_optional_fields.sql	Drops not null from products.sku, barcode, qr_code
20260802_profile_fields.sql	Adds profiles.mobile, profiles.avatar_url
20260802_profile_photos_bucket.sql	Re-declares the profile-photos bucket + policies (idempotent re-run)
20260802_stock_allocation_update.sql	Adds stock_entries.color, allocated_to_type, allocated_to
20260804_enquiries.sql	Creates the enquiries table, its order-id trigger, RLS policy, and the enquiry-images bucket
20260805_enquiries_hidden_flag.sql	Adds enquiries.hidden (soft-hide from lists without deleting)
20260807_enquiries_notes.sql	Adds enquiries.notes
20260808_enquiries_extracted_address.sql	Adds enquiries.extracted_address (OCR output)
20260810_brand_logo.sql	Adds brands.logo_url, creates the brand-logos bucket
20260811_backfill_product_sizes.sql	Data fix: renames a mis-named product, backfills missing S/M/L/XL/XXL product_sizes rows for 5 named core products

9. Edge Functions

9.1 low-stock-alert

File: `supabase/functions/low-stock-alert/index.ts`. A Deno-runtime HTTP function (rejects non-POST with 405). Using the service-role client (bypasses RLS), it selects every `product_sizes` row with `current_stock <= 0` joined to its product's `name` / `created_by`, builds one `notifications` row per result (type `out_of_stock`), and bulk-inserts them. Returns `{ inserted: N }`.

> This duplicates part of what `handle_stock_entry_after_insert()` already does reactively on every stock write — this function appears designed to run on a schedule (e.g. a daily cron via Supabase's scheduler) as a sweep/catch-all, though no scheduling configuration was found in the repository. Deploy command per `supabase/README.md`: `supabase functions deploy low-stock-alert`.

10. Frontend Application

The following inventory of pages, services, hooks, lib modules, components, and types was compiled by reading every listed file directly.

10.1 Pages — frontend/src/pages/*.tsx

LoginPage.tsx — Email/password login + registration (toggle between modes), Google OAuth button (currently non-functional pending provider setup, Sec. 6.3), forgot-password email flow. Uses react-hook-form + zod for both forms.

AboutPage.tsx / PrivacyPolicyPage.tsx / TermsPage.tsx / SupportPage.tsx — Static informational/legal pages, no data calls. SupportPage has a `tel:` link and a "Send Email" button that is a UI stub (no real email

is sent).

HomePage.tsx — The largest page (~1,820 lines) and the desktop dashboard / mobile home hub. Reads/writes the `enquiries` table via `services/enquiries.ts`. Drives orders through a fixed workflow — New → Giving for Printing → In Printing → Yet to Pack → Dispatch → Received — with per-stage advance buttons, an Order History panel (search/status/date filters, CSV and print-to-PDF export), a Delivered Orders sub-section, and per-order PDF receipt generation via jspdf. Fires `notifyInventorySync()` / `notifyOrderDelivered()` on status changes.

EnquiryPage.tsx — The order-intake form. Reads `fetchInventoryByBrand` (and, on desktop, `fetchBrands`); on save, writes a new `enquiries` row via `createEnquiry` and immediately deducts the requested stock via `deductStockForEnquiry`. Features customer-name autocomplete, a multi brand/size/quantity combination builder with live stock caps, a shipping-address screenshot upload with client-side OCR (tesseract.js) to extract editable address text, and TXT/PDF export of that address.

DashboardPage.tsx — "Stock Health Overview" — reads `fetchDashboardOverview()`. Shows a total-stock tile, four status tiles (healthy/warning/critical/out_of_stock — thresholds defined in `lib/stockStatus.ts`), a collapsible "Attention Needed" list, and an expandable per-product size grid. Auto-refreshes on the app's `INVENTORY_SYNC_EVENT` and on window focus.

StockUpPage.tsx — Simplified/fast stock-in flow for 3 fixed categories (Plain, Printed [Ceramic Shield/Sunkool/RS], Puma) with size-stepper inputs. Mirrors a log to localStorage (read by InventoryPage's snapshot calculation) and syncs to Supabase via `stockUpToSupabase()`, which auto-creates brand/category/product/product_size rows as needed.

InventoryPage.tsx — Mobile inventory snapshot. Does not query Supabase directly — computes a live snapshot in `lib/inventorySync.ts` by combining localStorage Stock Up entries with pending-enquiry deductions fetched from `services/enquiries.ts`.

ManagementPage.tsx — Simplified product master-data management (name/type/brand, auto-provisioned sizes). Reads/writes via `services/inventory.ts` (`fetchProducts`, `fetchBrands`, `createProduct`, `updateProduct`, `deleteProduct` [soft], `findOrCreateCategoryByName`). New products get all 5 standard sizes at creation.

ProductsPage.tsx — A fuller desktop product CRUD (category, brand, unit, minimum stock, purchase price, description, image upload, size checkboxes, status), including inline create-category/create-brand fallbacks and a link to `/brands` (not itself a defined route — see Sec. 5.1 gap note).

DistributorManagementPage.tsx / SupplierManagementPage.tsx — Standard CRUD screens (search, card/table view, edit, delete) against `distributors` / `suppliers` via `services/inventory.ts`. SupplierManagementPage is one of the orphaned/unrouted pages (Sec. 5.1).

OrderStatusPage.tsx — Desktop order-workflow manager: lists active (non-Received) orders, advances them one workflow stage at a time, shows status history, and can jump to `/track` with the tracking number stored via `lib/orderStorage.ts`.

TrackPage.tsx — Parcel/order tracking hub. Reads dispatched enquiries; on tracking-number entry, looks up and updates the matching enquiry (auto-marks "Received" when status is "delivered"). Also includes a fully local (localStorage-only) "New Shipment" logging form unrelated to the parcels table, and a link out to AfterShip.

DeliveredOrdersPage.tsx — Lists enquiries with `order_status === "Received"`; generates a branded PDF receipt per order via jspdf, previewed in an in-page iframe.

NotificationsPage.tsx — Merges persisted notifications (`lib/notificationsStorage.ts`, localStorage) with live low-stock alerts (`useLowStockAlerts()` hook, reading the notifications-driving query from `services/inventory.ts`).

MorePage.tsx — Mobile settings hub: links to Notifications/About/Privacy/Terms/Support/Logout, plus a "Reset All Stock" action (`resetInventoryToZero()`) confirmed via a modal dialog.

SettingsPage.tsx — App-wide settings. Dark-mode is persisted to Supabase (`app_settings` table via `services/settings.ts`) and synced with ThemeContext. All other sections (notification prefs, inventory defaults, data export, security, language) persist only to localStorage under `inventra_settings` ; the export buttons are UI stubs that show a success toast without producing a real file.

ProfilePage.tsx — Account settings: profile edit, photo upload (Storage bucket profile-photos), password/email change, mobile number update, theme toggle, and account deletion (rows in app_settings / profiles plus sign-out) — all via `services/auth.ts`.

AllocateStockPage.tsx / AllocationHistoryPage.tsx / ScannerPage.tsx / StockInPage.tsx / ParcelTrackingPage.tsx / ModulePage.tsx — Fully-built pages for stock allocation, allocation history, barcode/QR product+parcel lookup (queries Supabase directly rather than through a service module), a dedicated stock-receiving flow, and parcel creation/search — plus a generic placeholder (ModulePage, takes title/description props). None of these six are wired into routing (Sec. 5.1) and are not reachable in the running app.

10.2 Services — frontend/src/services/*.ts

File	Exports
auth.ts	login, register, fetchProfile, updateProfile, uploadProfilePhoto, changePassword, changeEmail, updateMobileNumber, deleteAccountData
inventory.ts	The largest module. Products/sizes/distributors/suppliers/categories/brands CRUD; fetchStockTransactions, fetchStockReceipts, fetchAllocationHistory, createStockTransaction, deductStockForEnquiry (idempotent, invoice-guarded), fetchInventoryByBrand, fetchLowStockAlerts, fetchDashboardOverview, stockUpToSupabase, resetInventoryToZero, createStockAllocation, fetchParcels / createParcel
enquiries.ts	fetchEnquiries, fetchPendingEnquiries, fetchDispatchedEnquiries, hideEnquiry, createEnquiry, updateEnquiryStatus, updateEnquiry, findEnquiryByTrackingNumber, updateEnquiryByTrackingNumber
dashboard.ts	fetchDashboardStats (counts + recent activity across products/distributors/suppliers/stock_entries/notifications/parcels); fetchStockTransactions; fetchAllocationList — stub, always returns []
analytics.ts	fetchAnalyticsOverview — daily/monthly/yearly stock-out totals, inventory value, top-10 products, parcel status breakdown, distributor receipt month-over-month pivot, top allocation recipients
settings.ts	fetchAppSettings, saveAppSetting (upsert against app_settings)

10.3 Hooks — frontend/src/hooks/*.ts

Hook	Purpose
useDashboard.ts	useQuery wrapper around fetchDashboardStats (key ["dashboard"])
useInactivityLogout.ts	30-minute auto-logout on user inactivity (Sec. 6.6)
useIsDesktopViewport.ts	Reactive matchMedia("(min-width: 1024px)") boolean used to branch mobile vs. desktop behavior (e.g. EnquiryPage's brand list)
useLowStockAlerts.ts	useQuery wrapper around fetchLowStockAlerts, invalidated on INVENTORY_SYNC_EVENT and window focus

10.4 Lib — frontend/src/lib/*.ts

File	Purpose
supabase.ts	Supabase client construction (Sec. 6.1), hasSupabaseConfig, getCurrentUser / getCurrentUserId
errors.ts	getErrorMessage() — normalizes a real Error or a Supabase/PostgREST error object into a readable string
inventorySync.ts	INVENTORY_SYNC_EVENT custom DOM event + notifyInventorySync() dispatcher (used app-wide after any stock-affecting write); calculateInventorySnapshot() (powers InventoryPage)
notificationsStorage.ts	localStorage-backed notification log: loadNotifications, saveNotification, notifyOrderDelivered, markNotificationRead, getUnreadCount; fires inventra:notifications-updated
orderStorage.ts	Shared order/enquiry types (OrderStatus, StatusHistoryEntry, RequirementCombination), the fixed ORDER_STATUSES workflow list, localStorage helpers for the "selected" tracking number
retryNetworkError.ts	retryOnNetworkError(fn, attempts=3, baseDelayMs=500) — retries only transient auth network failures
stockStatus.ts	StockStatus enum (out_of_stock/critical/warning/healthy) with fixed absolute thresholds — 500+ healthy, 200-499 warning, 1-199 critical, 0 out_of_stock — independent of each product's own minimum_stock
utils.ts	cn() — clsx + tailwind-merge class combiner used by all UI primitives

10.5 Components

components/dashboard/	StatCard.tsx (animated stat tile); TrendChart.tsx (Recharts area chart — currently renders hardcoded sample data, not live Supabase data)
components/shared/	EmptyState.tsx, GoogleIcon.tsx, InventraSearchGhostHero.tsx (decorative login animation), Skeleton.tsx, WarehouseScene.tsx (decorative SVG motifs)
components/ui/	button.tsx, card.tsx, confirm-dialog.tsx, input.tsx — shadcn-style primitives, styled via components.json (slate base color, CSS variables)

10.6 Types — frontend/src/types/index.ts

Shared TypeScript types: `DashboardStats`, `AnalyticsOverview`, `Product`, `ProductSize`, `Distributor`, `Supplier`, `Category`, `Brand`, `StockTransaction`, `Parcel`, `NotificationItem` — each mirrors the shape returned by its corresponding service function or database table.

11. Mobile Application (Expo)

`mobile/App.tsx` sets up a @react-navigation/native bottom-tab navigator with three tabs: Dashboard, Products, Scanner.

`mobile/src/services/api.ts` creates its own @supabase/supabase-js client from `EXPO_PUBLIC_SUPABASE_URL` / `EXPO_PUBLIC_SUPABASE_ANON_KEY`, talking to the same Supabase backend as the web app — there is no separate mobile API. As noted in Sec. 4.2, this client has `persistSession: false`, `autoRefreshToken: false`, and throws at import time if the env vars are absent.

Screen	Behavior
DashboardScreen.tsx	On mount, runs 5 parallel Supabase queries (counts for products/distributors/suppliers/parcels, all filtered `deleted_at is null`, plus a sum of `product_sizes.current_stock`) and renders them as stat cards; shows a hint message if queries fail or return nothing.
ProductsScreen.tsx	Fetches products (id, name, sku, selling_price) where `deleted_at is null`, newest first, in a FlatList with an empty-state message.
ScannerScreen.tsx	Uses expo-camera's useCameraPermissions / CameraView to request camera access and render a live scanner surface configured for qr, code128, ean13, upc_a — no scan-result handler is wired (no onBarcodeScanned callback present in source), so a successful scan currently does nothing.

12. Deployment

Compiled from `supabase/README.md`, `docs/DEPLOYMENT_GUIDE.md`, and `frontend/vercel.json` — all internally consistent with each other and with the actual code.

12.1 Supabase (database + auth + storage + edge functions)

1. Create a Supabase project.
2. Run `supabase/APPLY_ALL.sql` in the SQL Editor (idempotent, safe to re-run).
3. In Authentication → Providers → Email, disable "Confirm email" if instant post-signup login is wanted (the live project already has `mailer_autoconfirm: true`).
4. Verify the four storage buckets and their policies were created (Sec. 7.4).
5. Deploy the edge function: `supabase functions deploy low-stock-alert`.
6. Set edge function secrets: `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`.

12.2 Frontend (Vercel)

- Project root: `frontend`
- Build command: `npm run build` (runs `tsc -b && vite build`)

- Output directory: `dist`
- Environment variables: `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`
- `frontend/vercel.json` rewrites every path to `/index.html`, which is required for the client-side router's deep links (e.g. a direct hit on `/home`, including the OAuth redirect target) to resolve instead of 404ing.

12.3 Mobile (Expo)

- Environment variables: `EXPO_PUBLIC_SUPABASE_URL`, `EXPO_PUBLIC_SUPABASE_ANON_KEY`
- Local dev: `npm install && npm run start`
- Production builds: EAS (Expo Application Services) — referenced in the deployment guide, no EAS config file (`eas.json`) was found in the repository, so build profiles are not yet defined in-repo.

12.4 Outstanding setup step (as of this document)

Google OAuth is implemented in the frontend but the provider is switched off in the live Supabase project (confirmed via `/auth/v1/settings`, Sec. 6.3). To finish enabling it: create OAuth 2.0 credentials in Google Cloud Console with authorized redirect URI `{SUPABASE_URL}/auth/v1/callback`, then enable Google under Supabase Dashboard → Authentication → Providers with that Client ID/Secret, and ensure the app's redirect URLs (e.g. `https://<domain>/home`) are present in Authentication → URL Configuration → Redirect URLs.

13. Legacy / Stale Documentation in the Repository

Several existing docs describe an earlier, non-Supabase architecture and are inconsistent with the current codebase. They are called out explicitly here so they aren't mistaken for current guidance.

13.1 database/SCHEMA.md — stale

Lists tables named `auth_user`, `inventory_category`, `inventory_product`, `inventory_stocktransaction`, `parcels_parcel`, `reports_report`, etc. — Django app-prefixed table names. None of these exist in the current Supabase schema (Sec. 7.1 lists the real tables: profiles, products, stock_entries, parcels, ...). This file documents a prior Django/DRF backend that has been fully replaced.

13.2 docs/API_DOCUMENTATION.md, docs/SWAGGER.md, docs/postman_collection.json — self-aware, not stale in content

Unlike database/SCHEMA.md, these three files were updated to reflect the migration away from Django: API_DOCUMENTATION.md and SWAGGER.md both explicitly state "no Django REST server" / "no Swagger endpoints" and point to Supabase PostgREST + Auth + Edge Functions instead; the Postman collection has an empty item array with a note that "INVENTRA uses Supabase SDK directly (no custom REST backend)." They're accurate, just thin — there is no generated OpenAPI/Swagger surface to document because none exists (PostgREST's implicit REST surface over the tables in Sec. 7.1, governed by the RLS policies in Sec. 7.3, is the closest equivalent).

13.3 Evidence of the prior architecture

The root `.gitignore` still contains `backend/__pycache__/`, `backend/*.sqlite3`, `backend/db.sqlite3`, `backend/media/` — no `backend/` directory exists in the repository today, confirming a Python/Django backend was removed at some point and this cleanup line was simply never trimmed from .gitignore.

14. Known Gaps & Observations

Purely descriptive — things noticed while reading the code that don't fit elsewhere in this document, listed for awareness, not as recommendations.

- Google OAuth disabled server-side — frontend code is complete; the Supabase project's provider toggle is off (Sec. 6.3, Sec. 12.4).
- 7 unrouted pages — AllocateStockPage, AllocationHistoryPage, ScannerPage, StockInPage, SupplierManagementPage, ParcelTrackingPage, ModulePage exist but have no route/nav entry (Sec. 5.1).
- TrendChart.tsx shows hardcoded sample data (Jan-Jun), not a live query.
- dashboard.ts fetchAllocationList() is a stub that always returns [].
- SettingsPage export buttons are UI-only stubs — show a success toast, produce no file.
- ScannerScreen (mobile) has no scan-result handler — the camera surface renders but a successful scan is not acted on.
- Mobile Supabase client has no session persistence (persistSession: false, autoRefreshToken: false) — auth state does not survive an app restart.
- ProductsPage links to /brands, which is not a defined route in App.tsx (would fall through to the catch-all redirect to /home).
- All four Storage buckets are public-read with no per-object access control beyond URL obscurity (Sec. 7.4).
- No root package.json — frontend and mobile are fully independent npm projects; the root package-lock.json is an empty, effectively unused lockfile.
- No EAS config (eas.json) found for mobile production builds, despite the deployment guide referencing EAS.
- Two parallel "stock ledger" surfaces: the fast/simplified Stock Up flow (StockUpPage, mirrored to localStorage + stock_entries) and the fuller, unrouted Stock In flow (StockInPage, distributor/supplier-linked, stock_entries only) cover overlapping ground with different levels of detail.

End of document. Compiled by directly reading the INVENTRA repository's source files, SQL migrations, package manifests, and live Supabase Auth settings — no content here was inferred without a corresponding file or API response.